

# Föreläsning 260508 – Samlingar (Collections) II

## Arrayer

### Collections

Collections are objects used to group data.

They consist of a single object reference, and a series of values stored just like in variables.

You access all the values through the same object reference, using an index – an **identifier** that is unique to each value in the collection.

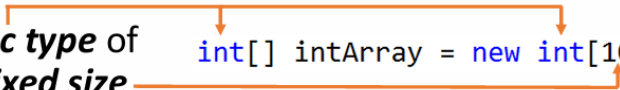
There are three main types of collection:

- Arrays
- Lists
- Dictionaries

### Collections - Arrays

Arrays are the most basic type of collection

They are defined for a **specific type** of variable, and always have a **fixed size**



```
int[] intArray = new int[10];
```

The diagram consists of two orange arrows. One arrow starts from the text 'specific type' and points down to the 'int[]' part of the code. The other arrow starts from the text 'fixed size' and points down to the '[10]' part of the code.

Arrays store values using a **0-based index** as an identifier. An array with 5 elements will have the indexes 0-4 for its elements.

Arrays offer good performance, but don't have many of the conveniences that other collections do. For example, they have to be **manually resized** when the amount of elements needs to change.

```
intArray[0] = 57;  
intArray[1] = 68;  
intArray[2] = 24;  
intArray[3] = 12;
```

# Collections - Multidimensional Arrays

Arrays can have more than one dimension - adding additional sets of indexes to store data in. Each combination of indexes will be unique, so each new dimension will increase the amount of potential values stored dramatically.

The amount of dimensions has to be defined when the array is created, and every time any value is referenced, an index must be included for each dimension

```
int[] intArray1 = new int[5];  
int[,] intArray2 = new int[5,5];  
int[, ,] intArray3 = new int[5,5,5];  
int[, , ,] intArray4 = new int[5,5,5,5];  
  
intArray1[3] = 12;  
intArray2[0,3] = 57;  
intArray3[1,2,5] = 68;  
intArray4[2,4,1,2] = 24;
```

Ändra storlek:

**Manuellt tillvägagångssätt:**

```
int[] gammalArray = { 1, 2, 3 };
```

```
int nyStorlek = 5;
```

```
// 1. Skapa en ny array med den nya storleken
```

```
int[] nyArray = new int[nyStorlek];
```

```
// 2. Kopiera befintliga värden
```

```
for (int i = 0; i < gammalArray.Length; i++)  
{  
    nyArray[i] = gammalArray[i];  
}
```

```
// 3. Peka om variabeln till den nya arrayen
```

```
gammalArray = nyArray;
```

## Rekommenderat sätt

```
int[] minArray = { 1, 2, 3 };
```

```
// Ändrar storleken till 5 och behåller befintliga värden
```

```
Array.Resize(ref minArray, 5);
```



## När använda multidimensionella arrayer

Använd `int[,]` när:

- **Prestandakritiska tillämpningar** (bildbehandling, spelrendering)
- **Matematiska beräkningar** (matrisoperationer)

// Exempel: Spelbräde 8x8

```
int[,] schackbräde = new int[8, 8];
```

```
schackbräde[0, 0] = 1; // Vit bonde
```